UNIVERSIDAD PRIVADA DEL NORTE

Facultad de Ingeniería

Ingeniería de Sistemas Computacionales

RETROFIT

Proyecto: "Lima en Acción"

Integrantes:

- Luz Mercedes Armijo Yomona (20%)
- Breider David Catashunga Peña (20%)
- Elias Alejandro Diaz Cahuas (20%)
- Nick Bryan Flores Huaman (20%)
- Anthony Jhoel Lozano Rubio (20%)

Curso: Desarrollo de Aplicaciones Móviles

Docente: Ing. Gerardo Sarmiento Quistan

San Juan de Lurigancho, Lima - Perú 2026

1. ¿Qué se hará?

Se definirá la persistencia de datos del negocio de "Lima en Acción" y el contrato entre la app y el backend. Esto incluye el diagrama entidad-relación (E-R), las clases (modelos) de la aplicación, las APIs con sus datos de entrada y de retorno, el avance del backend, el servicio y el tipo de base de datos elegidos, el lenguaje y framework del backend con su versión, y las consideraciones de seguridad.

¿En qué partes del proyecto se colocará?

- `feature_*/data/model/`: las clases (modelos) del dominio, como `Incident.kt`, `AuxiliaryContact.kt` y `EmergencyContact.kt`.
- `feature_*/data/remote/`: las fuentes remotas que consumen Firestore, Storage y las APIs del backend (por ejemplo `IncidentRemoteDataSource`).
- `feature_*/data/local/` y `core/data/local/`: las entidades y DAO de Room que replican el modelo como caché (PC-07).
- `functions/` (carpeta nueva en la raíz del repositorio): el código del backend en Cloud Functions, junto con `firestore.rules` y `storage.rules`.

¿Para qué es necesario?

- Para que app y backend compartan un único modelo de datos y no haya diferencias entre lo que se guarda en la nube y lo que se guarda en el teléfono.
- Para dejar definido el contrato de las APIs antes del Sprint 3 (Firebase y sincronización), de modo que los ViewModels y Repositories se conecten sin cambios de interfaz.
- Para cumplir los requerimientos de seguridad y privacidad (RNF-05, RNF-06 y RNF-07) con decisiones técnicas concretas y verificables.

2. Persistencia de datos del negocio

2.1 Estrategia de persistencia

La persistencia tiene tres niveles, coherentes con el PC-05 y el PC-07. Firestore es la fuente de verdad en la nube; SQLite (Room) es la copia local de la que siempre leen las pantallas; DataStore guarda las preferencias del usuario. Los reportes creados sin red se guardan en la tabla local `cola_envio` y se envían al volver la conexión.

Nivel	Tecnología	Qué guarda	Dónde vive
Nube (fuente de verdad)	Cloud Firestore	Usuarios, contactos de auxilio, reportes y directorio de emergencia	Firebase
Caché local (única fuente de lectura de la UI)	SQLite con Room	`usuario_cache`, `contacto_auxilio_cache`, `reporte_cache`, `contacto_emergencia_cache`, `cola_envio`	`core/data/local/` y `feature_*/data/local/`
Preferencias	Jetpack DataStore	Tema, idioma, radio de alertas, filtros de incidentes, contacto principal, etc.	`core/data/preferences/`
Archivos	Cloud Storage for Firebase	Foto de perfil y fotos de reportes (comprimidas a menos de 500 KB)	Firebase

2.2 Diagrama entidad-relación (E-R)

El diagrama muestra el modelo lógico del negocio. Aunque Firestore es una base documental, el modelo se describe con entidades y relaciones para facilitar su comprensión; en la sección 2.4 se indica cómo se traduce a colecciones de Firestore y a tablas de Room.

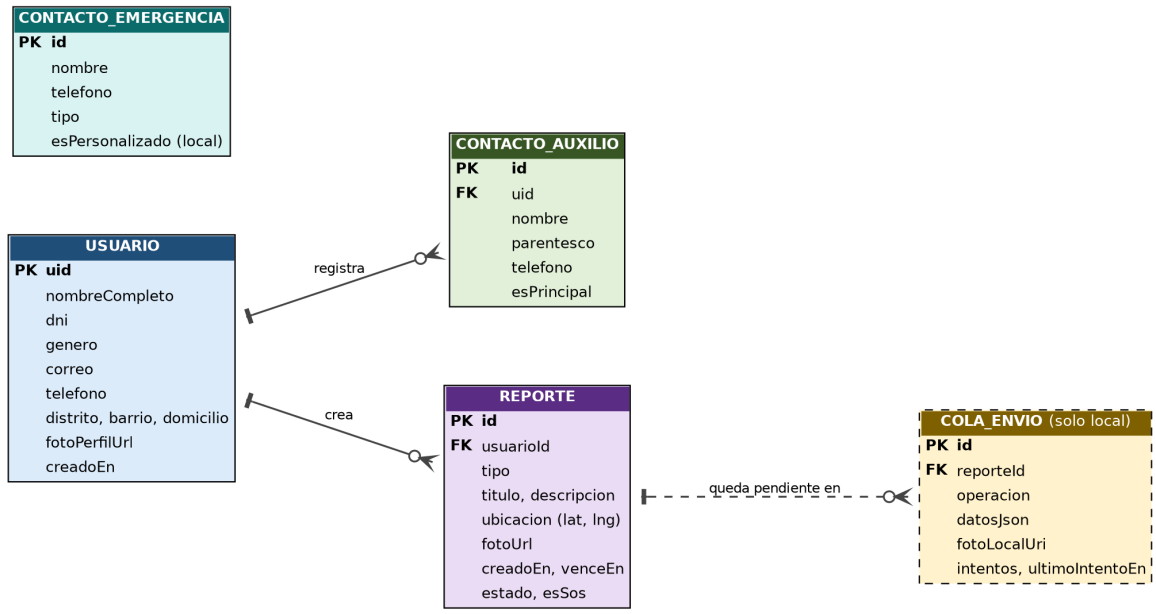


Figura 1. Diagrama E-R de Lima en Acción (| = uno, pata de gallo con círculo = cero o muchos).

Relación	Cardinalidad	Descripción
USUARIO registra CONTACTO_AUXILIO	1 a N	Un usuario puede registrar varios contactos de auxilio; cada contacto pertenece a un solo usuario (US-10, US-13).
USUARIO crea REPORTE	1 a N	Un usuario puede crear muchos reportes; cada reporte tiene un solo autor (US-03, US-12).
REPORTE queda pendiente en COLA_ENVIO	1 a N (solo local)	Un reporte creado sin red genera una o más operaciones pendientes hasta que se sincroniza (US-03, US-04).
CONTACTO_EMERGENCIA	Sin relación	Directorio de líneas oficiales (Policía 105, Bomberos 116, SAMU 106); es de consulta pública y no depende de un usuario (US-05).

2.3 Diccionario de datos

Los tipos se expresan en Firestore; en Room, `Timestamp` se guarda como `Long` (milisegundos) y `GeoPoint` como `latitud` y `longitud` (`Double`).

USUARIO (`usuarios/{uid}`)

Campo	Tipo	Obligatorio	Descripción
`uid`	string (PK)	Sí	Identificador de Firebase Authentication
`nombreCompleto`	string	Sí	Nombres y apellidos
`dni`	string (8)	Sí	Documento de identidad; dato sensible, nunca visible para otros usuarios

Campo	Tipo	Obligatorio	Descripción
genero	string (enum)	No	Valores de Gender
correo	string	Sí	Correo para el ingreso (validado)
telefono	string (9)	Sí	Celular validado por SMS
distrito, barrio, domicilio	string	No	Datos de residencia (US-11)
fotoPerfilUrl	string (URL)	No	Ruta en Cloud Storage
creadoEn	timestamp	Sí	Fecha de registro

CONTACTO_AUXILIO (usuarios/{uid}/contactosAuxilio/{id})

Campo	Tipo	Obligatorio	Descripción
id	string (PK)	Sí	Id del documento
uid	string (FK)	Sí	Usuario dueño; en Firestore queda implícito en la ruta
nombre	string	Sí	Nombre del contacto
parentesco	string (enum)	Sí	Valores de ContactRelationship
telefono	string (9)	Sí	Celular del contacto
esPrincipal	boolean	Sí	Contacto principal para el SOS (se refleja en DataStore: primaryContactName, primaryContactPhone)

REPORTE (reportes/{id})

Campo	Tipo	Obligatorio	Descripción
id	string (PK)	Sí	Id del documento (el mismo en la caché)
usuarioId	string (FK)	Sí	Autor; lo asigna el backend desde el token, no el cliente
tipo	string (enum)	Sí	Valores de ReportType
titulo, descripcion	string	Sí / No	Texto corto del aviso
ubicacion	GeoPoint	Sí	Ubicación pública con desplazamiento aleatorio de unos 50 m (RNF-07)
fotoUrl	string (URL)	No	Foto comprimida (menos de 500 KB) en Storage
creadoEn, venceEn	timestamp	Sí	venceEn = creadoEn + 12 h (US-06)
estado	string (enum)	Sí	PENDIENTE_ENVIO, PUBLICADO o VENCIDO
esSos	boolean	Sí	Indica un pin rojo crítico (US-04)

CONTACTO_EMERGENCIA (`directorioEmergencia/{id}`) y COLA_ENVIO (solo local)

Entidad	Campos	Notas
CONTACTO_EMERGENCIA	`id` (PK), `nombre`, `telefono`, `tipo`; en local además `esPersonalizado`	Se precarga en la caché para funcionar sin red y sin sesión (US-05, RNF-03)
COLA_ENVIO	`id` (PK), `reporteId` (FK), `operacion`, `datosJson`, `fotoLocalUri`, `intentos`, `ultimoIntentoEn`	No existe en Firestore; hasta 5 intentos con espera creciente (RNF-04)

2.4 Mapeo entre E-R, Firestore y Room

Entidad E-R	Clase Kotlin	Colección Firestore	Tabla Room
USUARIO	`Usuario` (perfil)	`usuarios/{uid}`	`usuario_cache`
CONTACTO_AUXILIO	`AuxiliaryContact`	`usuarios/{uid}/contactosAuxilio/{id}`	`contacto_auxilio_cache`
REPORTE	`Incident` / `ReportHistoryItem` (vista)	`reportes/{id}`	`reporte_cache`
CONTACTO_EMERGENCIA	`EmergencyContact`	`directorioEmergencia/{id}`	`contacto_emergencia_cach`
COLA_ENVIO	`ColaEnvio`	No aplica (solo local)	`cola_envio`

2.5 Reglas de integridad

- Un documento de `contactosAuxilio` solo existe dentro de un usuario; al eliminar la cuenta se eliminan sus contactos y reportes.
- Solo un contacto del usuario puede tener `esPrincipal = true`.
- Los reportes vencidos dejan de mostrarse en el mapa (consulta con `venceEn` mayor a la hora actual), pero se conservan para el historial de su autor (US-12).
- El `id` del reporte lo genera el cliente (UUID) al crearlo; así un reintento de la cola no duplica el reporte.

3. Clases (modelos)

Los modelos viven en `feature_*/data/model/` y son clases de datos de Kotlin independientes de Room y Firestore. Cada fuente de datos las convierte con funciones de mapeo (`toEntity()`, `toDomain()`), de modo que la capa de presentación nunca conoce el formato de almacenamiento.

Clase	Paquete	Atributos principales
`Usuario`	`feature_profile/data/model`	`uid`, `nombreCompleto`, `dni`, `genero: Gender`, `correo`, `telefono`, `distrito`, `barrio`, `domicilio`, `fotoPerfilUrl`
`AuxiliaryContact`	`feature_profile/data/model`	`id`, `nombre`, `parentesco: ContactRelationship`, `telefono`, `esPrincipal`
`Incident`	`feature_alerts/data/model`	`id`, `usuarioId`, `tipo: ReportType`, `titulo`, `descripcion`, `latitud`, `longitud`, `fotoUrl`, `creadoEn`, `venceEn`, `estado: EstadoReporte`, `esSos`

Clase	Paquete	Atributos principales
EmergencyContact	feature_emergency/data/model	id, nombre, telefono, tipo, esPersonalizado
ColaEnvio	core/data/local	id, reporteId, operacion, datosJson, fotoLocalUri, intentos, ultimoIntentoEn
Enumeraciones	data/model	Gender, ContactRelationship, ReportType (ya existen) y EstadoReporte (PENDIENTE_ENVIO, PUBLICADO, VENCIDO), OperacionCola (CREAR_REPORTE, ACTIVAR_SOS)

4. APIs: datos de entrada y retorno

La app usa dos mecanismos. Las lecturas y escrituras simples del propio usuario van directo con el SDK de Firebase, protegidas por reglas de seguridad. Las operaciones con lógica sensible (crear reportes, activar el SOS) pasan por funciones HTTPS callable del backend, que validan los datos, asignan el autor desde el token y aplican la privacidad de la ubicación. Todas las funciones se publican en una región cercana a Lima (por ejemplo `southamerica-east1`) y exigen un usuario autenticado.

4.1 Autenticación (SDK de Firebase Authentication)

Operación	Entrada	Retorno	Historia
Validar teléfono (registro)	telefono en formato +51XXXXXXXXX	verificationId; Firebase envía un código de 6 dígitos por SMS	US-01
Confirmar código SMS	verificationId, codigo (6 dígitos)	Usuario autenticado (uid) y token de ID	US-01
Ingresar	correo, clave	Usuario autenticado y token de ID (expira en 1 h)	US-01
Recuperar clave	correo; luego el código recibido y claveNueva	Confirmación de la clave actualizada	US-09
Cerrar sesión	Ninguna	Sesión cerrada	US-14

4.2 APIs de negocio

ID	Operación	Mecanismo	Entrada	Retorno	Historia
API-01	Crear reporte	Función callable crearReporte	id, tipo, titulo, descripcion?, latitud, longitud, fotoPath?	id, estado, creadoEn, venceEn	US-03, US-07
API-02	Activar SOS	Función callable activarSos	id, latitud, longitud	id, estado, venceEn, contactosAvisados	US-04
API-03	Mapa y alertas en tiempo real	Escucha de Firestore sobre reportes	venceEn > ahora, filtros de tipo	Lista de reportes públicos (con coordenadas desplazadas) que alimenta reporte_cache	US-02, US-06

ID	Operación	Mecanismo	Entrada	Retorno	Historia
API-04	Mis reportes	Consulta de Firestore	`usuarioId` = `uid`, orden por `creadoEn` descendente	Lista de reportes con su estado	US-12
API-05	Listar contactos de auxilio	Consulta de Firestore	`uid`	Lista de `AuxiliaryContact`	US-13
API-06	Agregar contacto de auxilio	Escritura en Firestore	`nombre`, `parentesco`, `telefono`, `esPrincipal`	`id` del contacto creado	US-10, US-13
API-07	Eliminar contacto de auxilio	Eliminación en Firestore	`id`	Confirmación	US-13
API-08	Obtener perfil	Lectura en Firestore	`uid`	Documento `Usuario`	US-11
API-09	Actualizar perfil	Escritura en Firestore	`nombreCompleto?`, `correo?`, `distrito?`, `barrio?`, `domicilio?`	Perfil actualizado	US-11
API-10	Subir foto de perfil	Cloud Storage (`perfiles/{uid}/foto.jpg`)	Archivo de imagen menor a 500 KB	`fotoPerfilUrl`	US-11
API-11	Subir foto de reporte	Cloud Storage (`reportes/{uid}/{id}.jpg`)	Archivo de imagen menor a 500 KB	`fotoPath` para enviar en API-01	US-03, US-08
API-12	Directorio de emergencia	Lectura de Firestore (pública)	Ninguna	Lista de `EmergencyContact`	US-05

4.3 Errores comunes

Código	Cuándo ocurre	Qué hace la app
`unauthenticated`	No hay sesión o el token venció	Pide iniciar sesión de nuevo
`invalid-argument`	Datos fuera de rango (por ejemplo coordenadas inválidas o título vacío)	Muestra el error de validación en el formulario
`permission-denied`	La regla de seguridad rechaza la operación	Muestra un mensaje de acceso no permitido
`resource-exhausted`	Se superó el límite de reportes permitidos por hora	Informa el límite y el tiempo de espera
`unavailable`	Sin red o servicio no disponible	Guarda en `cola_envio` y reintenta (hasta 5 veces, espera creciente)

5. Servicio a usar

Se usará Firebase como servicio de backend (BaaS) de Google. Evita montar y mantener un servidor propio, se integra de forma nativa con Android y Kotlin, y ya forma parte del plan del proyecto (Sprint 3).

Servicio de Firebase	Uso en Lima en Acción	Historias
Authentication	Registro con validación de teléfono por SMS, ingreso con correo y clave, recuperación de clave	US-01, US-09
Cloud Firestore	Usuarios, contactos de auxilio, reportes y directorio de emergencia, con escucha en tiempo real	US-02 a US-06, US-10 a US-13
Cloud Storage for Firebase	Fotos de perfil y de reportes ya comprimidas	US-07, US-08, US-11
Cloud Functions for Firebase	Lógica de servidor: crear reportes, activar el SOS, privacidad de ubicación, límites de uso	US-03, US-04, US-07
App Check	Verifica que las solicitudes provienen de la app original	Seguridad
Firebase Emulator Suite	Pruebas locales de reglas y funciones sin tocar datos reales	Pruebas

6. Tipo de base de datos

Aspecto	Decisión
Base en la nube	Base de datos NoSQL orientada a documentos: Cloud Firestore en modo nativo, con colecciones y subcolecciones
Por qué NoSQL	Escucha en tiempo real hacia la app, soporte sin conexión, crecimiento automático y consultas simples por documento; el modelo del proyecto no requiere uniones complejas
Base local	SQLite (relacional) con Room, como caché y única fuente de lectura de la interfaz
Preferencias	Jetpack DataStore (clave-valor)
Región	Una región de Sudamérica (por ejemplo `southamerica-east1`) para reducir la latencia desde Lima

7. Lenguaje y framework del backend (versiones)

Componente	Tecnología	Versión
Backend (lógica de servidor)	Cloud Functions for Firebase, 2.ª generación	SDK `firebase-functions` 6.x
Lenguaje del backend	TypeScript sobre Node.js	Node.js 22 (LTS), TypeScript 5.x
Acceso a servicios desde el servidor	Firebase Admin SDK	`firebase-admin` 13.x
Herramientas	Firebase CLI y Emulator Suite	Última versión estable
App móvil	Kotlin con Jetpack Compose; Room, DataStore y Firebase BoM	Las definidas en `build.gradle.kts` del proyecto

8. Avance del backend

Componente	Estado	Detalle
Modelo de datos (colecciones de Firestore y tablas de la caché)	Completado	Definido en la sección 6 del PC-05 y refinado en este documento
Diagrama E-R, clases y contrato de APIs	Completado	Secciones 2 a 4 de este documento
Caché local: Room y DataStore en la app	En curso	Sprint 2 (PC-07): reemplazo de los repositorios Mock por repositorios con Flow
Proyecto Firebase y colecciones de Firestore	En curso	Tarea del Sprint 2: proyecto creado y colecciones diseñadas
Reglas de seguridad de Firestore y Storage	Borrador	Definidas en la sección 9
Firebase Authentication (SMS, correo y clave)	Pendiente	Sprint 3 (US-01)
Cloud Functions: `crearReporte` y `activarSos`	Pendiente	Sprint 3 (US-03, US-04)
Escucha de Firestore hacia la caché (mapa y alertas)	Pendiente	Sprint 3 (US-02)
Cloud Storage para fotos	Pendiente	Sprint 3 (US-07)
Sincronización de `cola_envio` con reintentos	Pendiente	Sprint 3 (US-04)
Pruebas de reglas, funciones, ViewModels y DAO	Pendiente	Sprint 3

9. Consideraciones de seguridad

Área	Riesgo	Medida	Req.
Autenticación	Cuentas falsas o abuso del envío de SMS	Validación por SMS con Firebase Authentication, App Check y límites de uso; sesión con token que expira en 1 hora	RNF-05
Autorización	Acceder o modificar datos de otro usuario	Reglas de Firestore y Storage por `uid`; los reportes se crean solo desde Cloud Functions, que toman el autor del token	RNF-05
Cifrado en tránsito	Intercepción de datos	Solo conexiones HTTPS/TLS (valor por defecto de Firebase); sin tráfico en texto plano	RNF-06
Contraseñas y secretos	Claves expuestas	Firebase Authentication guarda la clave con hash; nunca se guardan claves en DataStore ni SQLite; los secretos del servidor van en Secret Manager y no en el repositorio	RNF-06
Privacidad de ubicación	Revelar dónde vive o está quien reporta	El pin público se desplaza unos 50 m de forma aleatoria; la ubicación exacta se guarda en un documento privado visible solo para el autor	RNF-07
Datos personales	Exposición del DNI, teléfono y domicilio	Principio de mínimo dato; el perfil solo lo lee su dueño; el consentimiento y la finalidad se informan al registrarse, conforme a la Ley N.° 29733	RNF-07
Fotos	Archivos pesados, no válidos o con metadatos de ubicación (EXIF)	Storage solo admite imágenes menores a 500 KB; al comprimir se eliminan los metadatos EXIF	RNF-10

Área	Riesgo	Medida	Req.
Validación de entrada	Datos manipulados desde un cliente alterado	Las validaciones del PC-06 se repiten en el servidor (tipos, rangos de coordenadas, longitudes)	-
Abuso del SOS y de reportes	Alertas falsas o saturación	Límite de reportes por usuario y hora (`resource-exhausted`), registro de eventos para auditoría, App Check	RNF-09
Datos en el teléfono	Extraer la base SQLite desde una copia de respaldo	`android:allowBackup="false"`; no guardar datos innecesarios en la caché (por ejemplo el DNI); evaluar cifrado de la base si se amplía el alcance	RNF-06
Cuenta y retención	Datos conservados sin necesidad	Al eliminar la cuenta se borran perfil, contactos, reportes y fotos; los reportes vencidos dejan de ser públicos	RNF-07
Registros	Datos personales en los logs	No registrar DNI, teléfonos ni coordenadas exactas en Cloud Logging	-